Symbolic Data Classification and Data-Reference Handling in Zone

This report documents two things:

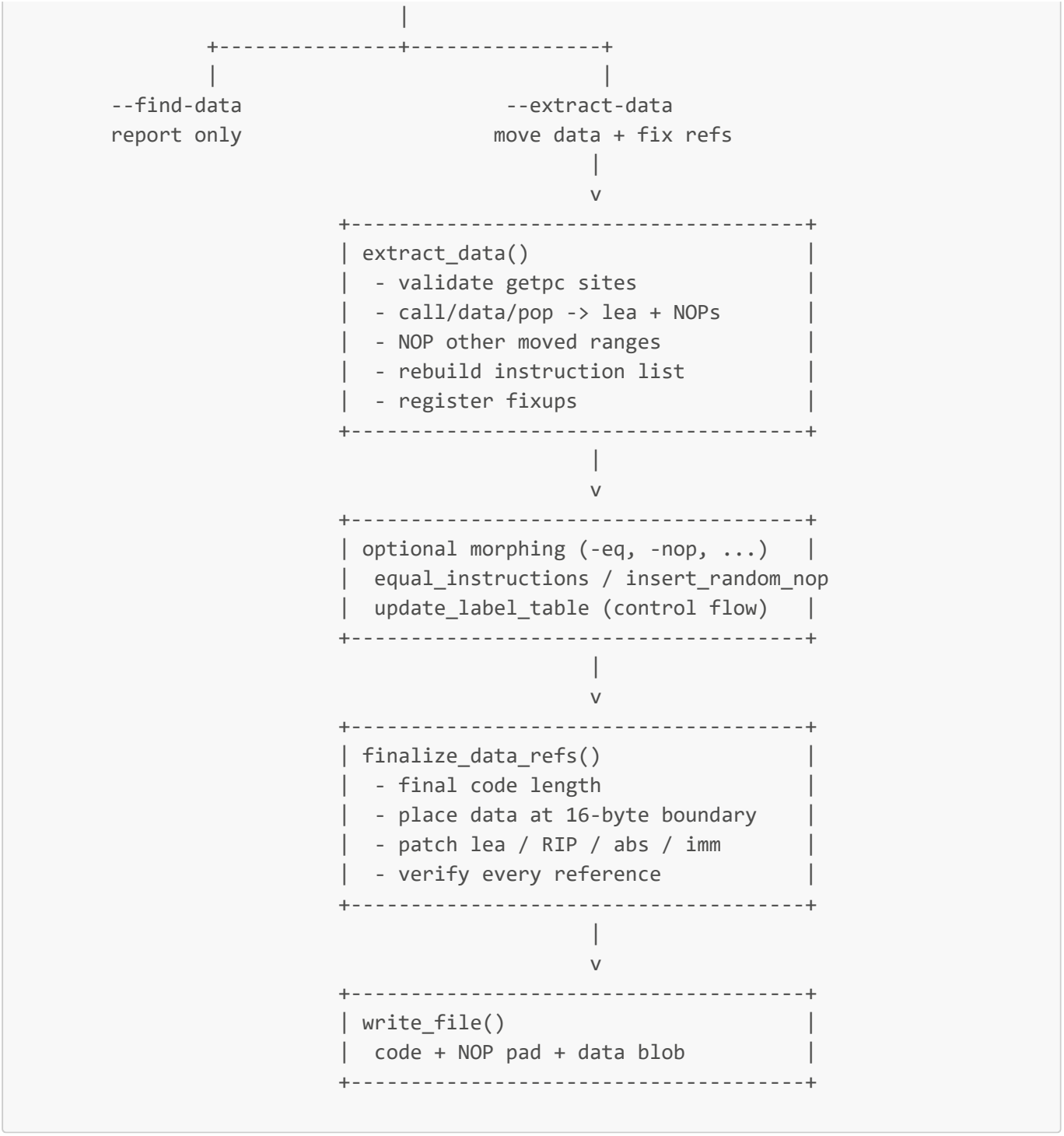
1. how `symbolic_payload_coverage.py` uses symbolic execution to classify bytes as data candidates;
2. how the metamorphic engine in `pobf.py` treats that data, how data references are discovered, and how they are rewritten/patch after the optional morphing.

The report describes the implementation as it exists now, including the size-preserving extraction added for `--extract-data`.

---

0. Pipeline overview

```
                raw payload bytes
                       |
                       v
+-----------------------------------------+
| symbolic_payload_coverage.py            |
|                                         |
|  1. CFGFast (static code candidates)    |
|  2. bounded symbolic exploration        |
|     (observed instruction addresses)    |
|  3. second CFGFast from observed entries|
|  4. classify bytes                      |
|     - executed                          |
|     - code / cfg_data                   |
|     - strings (ascii / utf16le)         |
|     - raw ranges (cfg_data/outside_cfg) |
+-----------------------------------------+
                       |
                       v
          sections = [{address, offset,
                       length, kind}, ...]
                       |
                       v
+-----------------------------------------+
| pobf.Zone.find_data_and_refs()          |
|                                         |
|  static scan of the linear-sweep        |
|  instructions:                          |
|    - immediates (non control-flow)      |
|    - RIP-relative / absolute memory     |
|    - call $+N; pop reg  (getpc)         |
+-----------------------------------------+
                       |
                       v
             xrefs = [(source, target, len)]
```



The two halves are deliberately separated:

- the symbolic engine answers **"which bytes look like data?"**;
- the metamorphic engine answers **"which instructions refer to those bytes, and how do we keep the references correct when the code changes?"**

The symbolic engine never emits xrefs. The xref scan is static and works on the linear-sweep instruction list. This separation is the reason the original implementation found the messagebox strings but could not find any xrefs to them.

---

1. Symbolic execution for data classification

1.1 Model and environment

`find_data_sections()` builds a raw `blob` angr project:

- architecture `x86`/`amd64`;
- entry point `base + entry_offset`;
- `auto_load_libs=False`;
- initial state is a `blank_state` with `SYMBOL_FILL_UNCONSTRAINED_MEMORY` and `SYMBOL_FILL_UNCONSTRAINED_REGISTERS` (`symbolic_payload_coverage.py:388`, `explore`).

This is a deliberately empty OS environment: there is no TEB, no PEB, no `kernel32!LoadLibraryA`, no imports, no stack arguments. Consequently the analysis can observe instruction addresses but cannot execute real Windows API semantics. That is the central limitation behind the "coverage is provisional" wording in every report.

1.2 Static pass: CFGFast as a code prior

Before any symbolic work, `static_candidates()` runs CFGFast (`symbolic_payload_coverage.py:87`):

```
CFGFast(
    start_at_entry=True,
    force_complete_scan=scan_all,
    force_smart_scan=False,
    function_prologues=False,
    symbols=False,
    normalize=True,
    data_references=True,
    function_starts=sorted(function_starts) if function_starts else None,
)
```

From the recovered graph it extracts:

Output	Meaning
`addresses`	instruction start addresses belonging to CFG nodes
`unclassified`	byte ranges outside every CFG node (`ranges_not_in_cfg`)
`code_spans`	`(node.addr, node.addr + node.size)` for every node
`inferred_data`	`cfg.model.memory_data` entries (CFG-inferred data objects)

`data_references=True` is important: CFGFast tries to infer data objects from memory operands. That helps classify jump tables and similar data, but it is also the source of the low-base failure described in section 1.8.

The CFG is only a **prior**. CFGFast is a linear/smart scanner, not an execution proof: it can recover data as instructions (the messagebox strings are a perfect example), and it can miss code reached only through indirect transfers.

1.3 Symbolic pass: bounded exploration

`explore()` (`symbolic_payload_coverage.py:388`) is a manual state loop rather than `simgr.explore()`:

- one state at a time is taken from a FIFO `pending` deque;
- `simgr.step()` advances that state;
- only `active` states are retained; `deadended`, `unconstrained` and `errored` states are counted and discarded;
- an `instruction` breakpoint (`BP_BEFORE`) records `state.scratch.ins_addr` into `observed` when it is inside the payload, or into `external` otherwise;
- limits are checked between steps: `max_steps`, wall-clock `timeout`, `max_queued_states`;
- each state's solver query timeout is clamped to the remaining exploration budget (`state.solver._solver.timeout`).

The result is a set of **observed instruction addresses**, not a memory trace. There is no `mem_read`/`mem_write` breakpoint and no register-value log. The engine therefore knows which instructions ran, not which data was touched.

`stop_reason` is one of `frontier_exhausted`, `max_steps`, `timeout`, `max_queued_states`. In practice Windows shellcode that walks the PEB usually ends in `timeout` or in load-concretization errors; the report exposes this via `pending_states`, `unconstrained_states`, `errors`, and the `provisional` warning.

1.4 Byte masks

`identify_data()` (`symbolic_payload_coverage.py:119`) reduces the run to four byte masks:

```
code[offset]       : byte belongs to a recovered CFG block
executed[offset]   : byte was decoded as part of an observed instruction
cfg_data[offset]   : byte belongs to a CFG-inferred data object (memory_data)
string_bytes[offset] : byte already consumed by a detected string
```

`code` comes from `code_spans`; `cfg_data` from `inferred_data`; `executed` from disassembling the raw bytes at each observed address and marking the instruction size. This last detail matters: an observed address marks the whole instruction, not just its first byte.

1.5 String classification

Two regular expressions are used (`symbolic_payload_coverage.py:179`):

```
("ascii",   rb"[\x20-\x7e]{4,}\x00")
("utf16le", rb"(?:[\x20-\x7e]\x00){4,}\x00\x00")
```

A match is kept only if:

1. none of its bytes are in `executed`;
2. none of its bytes are already in `string_bytes`;
3. if it overlaps `code`, the exact `(start, end)` pair is a **call/pop gap** (see below).

The third rule is the important one. A forward `call` to a `pop` instruction can jump over inline data:

```
call 0xea          ; pushes 0xdf, jumps to pop
0xdf: "user32.dll\0"
0xea: pop rcx      ; rcx = 0xdf
```

But a normal function call can also skip over ordinary code and return into the gap. The engine therefore requires the string to fill the **entire** gap `(call.end, call_target)`, and requires no gap byte to be executed:

```
gap_start = address + instruction.size
target_offset = instruction.operands[0].imm - base
if target_instruction.mnemonic == "pop" and not any(executed[gap_start -
base:target_offset]):
    call_pop_gaps.add((gap_start - base, target_offset))
```

For the messagebox payload this is exactly true for all three strings, so they survive even though CFGFast decoded them as instructions and marked `cfg_code_overlap=True`.

String sizes include the terminating NUL. The string record carries `kind`, offsets, size, text, hex, and `cfg_code_overlap`; the section wrapper only exposes `address`, `offset`, `length`, `kind`.

1.6 Raw ranges

After strings, the engine walks the payload and groups contiguous bytes that are:

- not executed;
- not consumed by a string;
- not code, **or** code that CFGFast tagged as `cfg_data`.

The group kind is `cfg_data` if the first byte is in `cfg_data`, otherwise `outside_cfg` (`symbolic_payload_coverage.py:206`). `outside_cfg` is the least reliable category: it means "unexecuted and not recovered by CFGFast", which can be data, padding, or genuinely unreachable code. For this reason the extractor skips `outside_cfg` unless `--extract-unknown` is passed.

1.7 Second CFG pass

Indirect transfers can enter code that the first CFGFast pass missed. After the symbolic run, the engine looks for observed block entries inside `unclassified` ranges and uses the smallest entry of each range as an extra `function_starts` seed for a second CFGFast pass (`symbolic_payload_coverage.py:339-350`). On the messagebox payload this is what recovers the resolver body at `0xd` after the initial CFG only found 11 blocks.

This pass is why the CFG can cover code that the symbolic run did not execute: the seed comes from one observed entry, and CFGFast then follows the usual direct branches and fall-through.

1.8 Address space and the low-base rebasing

The public API accepts `base_address` and returns addresses in that space. The implementation, however, refuses to map the blob at a very low base (`symbolic_payload_coverage.py:319`):

```
analysis_base = base_address if base_address >= 0x10000 else 0x400000
shift = base_address - analysis_base
```

All analysis calls (`angr.Project`, `static_candidates`, `explore`, `identify_data`) use `analysis_base`; string/error addresses are translated back by `shift`, and sections are built with the requested `base_address`.

There are two concrete reasons.

CFGFast data-pointer aliasing. `data_references=True` can interpret a small constant inside the payload as an absolute pointer. The messagebox payload contains `movzx rcx, word ptr [rdx+0x4a]`. At base 0, `0x4a` is a valid address inside the blob; CFGFast marks it as data, truncates the block at `0x4a`, and everything after it falls out of the CFG. At base `0x400000` the constant `0x4a` is outside the mapped region, so the heuristic does not fire. Measured on the sample:

```
base=0x0      second CFG pass: 16 blocks, code stops at 0x4a
base=0x1000   second CFG pass: 26 blocks
base=0x400000 second CFG pass: 26 blocks
```

The immediate consequence for classification: un-recovered code becomes `outside_cfg`, and a later consumer can mistake code for data.

Payload/PEB aliasing. At base 0 the payload is mapped at offset 0. The PEB read `mov rdx, gs:[rdx+0x60]` therefore reads address `0x60`, which is inside the payload, instead of uninitialized memory. The run can then explore on a garbage concrete "PEB" value. At a nonzero base the same read is unconstrained symbolic memory, and the run stops at the concretization failure. The deep 6459-step run at base 0 was therefore an artifact, not better coverage.

The standalone CLI still maps at the requested base and prints a warning for bases below `0x10000`; only the importable `find_data_sections()` performs the internal rebasing.

1.9 What the classification guarantees

The output is explicitly **candidate data**, not a proof:

- `executed` bytes are trusted as code (they were decoded as executed instructions);
- CFG code is trusted as code unless an exact call/pop gap says otherwise;
- strings are a structural heuristic (printable + NUL, optional call/pop corroboration);
- `cfg_data` comes from CFGFast's data-reference inference;
- `outside_cfg` is a residual category and may be code.

Every result should be read as "the analysis has evidence for data here", never "this is data". The report includes `coverage_status`, `stop_reason`, `pending_states`, `unconstrained_states`, and `errors` precisely so the consumer can see how strong that evidence is.

1.10 Why no xrefs come out of the symbolic engine

`explore()` records only `insn_addr`. It does not record:

- memory-read addresses (`state.inspect.b("mem_read", ...)`);
- memory-write addresses;
- register values at API-call sites;
- the value pushed by `call`.

The symbolic engine can report that a byte was not executed during the bounded run; it cannot, by itself, tell which instruction formed a pointer to it. That is exactly why the messagebox run printed three strings and zero useful xrefs in the original implementation.

---

2. Data in the metamorphic engine

2.1 Instruction model

`Zone.__init__` reads the payload and performs a linear sweep at address 0 (`pobf.py:38` onward):

```
self.cs = Cs(CS_ARCH_X86, CS_MODE_64)
self.cs.detail = True
self.cs.skipdata = True
for i in self.cs.disasm(self.raw_code, 0):
    instruction = Instruction(i, i, i, None, None)
    self.instructions.append(instruction)
    self.instr_dict[i.address] = instruction
```

Each `Instruction` object keeps four Capstone views:

Field	Role
`original_instruction`	first decoding, used for analysis
`old_instruction`	instruction before the latest substitution
`new_instruction`	current bytes/address
`prev_instruction` / `next_instruction`	list neighbours

`rebuild_instructions_from_bytes()` (`pobf.py:1160`) can replace the whole list after a byte-level edit; it re-disassembles and rebuilds `instr_dict`, `updated_instr_dict`, `short_label_table`, and the neighbour links.

2.2 The layout model

There is no explicit segment or section object for a raw payload. The layout is implicit:

```
output bytes = concatenation of instr.new_instruction.bytes, in list order
```

The address of an instruction is not authoritative. `update_instr()` recomputes each address from the previous instruction:

```
addr = prev.new_instruction.address + len(prev.new_instruction.bytes)
```

after every size-changing morph. The output file is then the concatenation of the current bytes (`write_file`, `pobf.py:2588`).

This is why **size-preserving edits are attractive**: if no instruction changes size, every address remains valid and the label table keeps working.

2.3 Data is not first-class

Originally, the engine treated the whole payload as instructions. Inline data was whatever the linear sweep happened to decode; there was no data object. The only data-related state was:

```
self.data_xrefs = []
self.data_sections = []
```

added for the analysis. Data bytes were not moved, not protected, and not distinguished from code by the morphing functions.

With the new opt-in extraction, data becomes an explicit artifact:

```
self.extracted_data = [
    {"offset": ..., "length": ..., "bytes": ..., "kind": ...,
     "new_address": ...},
]
self.data_ref_fixups = [
    {"kind": "getpc"|"rip"|"abs"|"imm",
     "instr": <Instruction object>, "section": <index>, "delta": ...,
     ...},
]
```

The `data_ref_fixup` is also attached to the `Instruction` wrapper. This distinction matters: morphing functions usually replace `instr.new_instruction` (the Capstone object) but keep the `Instruction` wrapper, so a fixup that references the wrapper survives address changes.

2.4 Control-flow relocation vs. data relocation

These are two different mechanisms and must not be confused.

Control flow (`pobf.py:203`, `:290`):

1. `create_label_table()` builds `(branch_instruction, target_instruction)` pairs for direct `jmp`/`call`/`loop`/`jrcxz`.
2. Morphing changes instruction sizes and order.
3. `update_instr()` recomputes addresses sequentially.

4. `update_label_table()` re-encodes each branch so its operand equals `target_instruction.new_instruction.address`. If a jump grows, it inserts NOPs or re-iterates until stable.

Data references (`pobf.py:1609`, `finalize_data_refs()`):

1. Extraction rewrites a data site to a placeholder `lea reg, [rip + 0]` and records a fixup.
2. Morphing moves the instruction and changes the total code length.
3. `finalize_data_refs()` computes the *final* address of every instruction from the byte list, lays out the data at the end of the final code, and only then fills in the displacement/immediate.
4. Every patched reference is decoded and checked.

The split exists because the displacement of a data reference depends on the final total code length, which is not known until all morphing is done. A control-flow label, by contrast, only needs the final address of another instruction, which the label table already tracks.

---

3. How data references are found

`find_data_and_refs()` (`pobf.py:985`) is a static scan over `self.instructions`. It takes data ranges (normally from `find_data_sections()`) and returns tuples:

```
(source_instruction_address, target_data_address, matched_candidate_length)
```

3.1 Preconditions

- Instructions and sections must use the same address space. `Zone` always disassembles at 0, so the expected `base_address` is 0.
- `sections` is a list of dicts with integer `address` and `length`.
- Only instructions present in the linear sweep are considered. This is a limitation when Capstone stops early (see section 5).

3.2 Scan rules

For every instruction:

1. Skip `id == 0` and `.byte` entries (Capstone skipdata placeholders).
2. Skip the instruction if its byte range overlaps any data range. This is the fix for the original false positive: the byte at `0xe1` inside `"user32.dll"` decodes as `jb 0x117`, whose branch target landed inside `"MessageBox"`. That instruction is data, not code, and must not generate a reference.
3. Compute `is_control_flow` from the Capstone groups and mnemonics: `X86_GRP_JUMP`, `X86_GRP_CALL`, `X86_GRP_BRANCH_RELATIVE`, `loop`, `loope`, `loopne`, `jrcxz`, `jecxz`, `jcxz`.
4. **getpc**: if the instruction is `call` with an immediate target, decode the callee directly from the original bytes. If it is a `pop`, the pushed return address `insn.address + insn.size` is added as a data reference. This is the key addition for position-independent shellcode.
5. **Explicit operands**:
   - `X86_OP_IMM`: candidate unless the instruction is control flow. Branch and call targets are code addresses, not data pointers.

    - `X86_OP_MEM`:
        - skip FS/GS unless the instruction is `lea` (segment base unknown at static time);
        - skip indexed operands (register value unknown);
        - `RIP`-relative: `insn.address + insn.size + disp`;
        - `EIP`-relative: same with a 32-bit mask;
        - absolute (`base == invalid`): `disp & address_mask`.
6. `add_xref()` only records the target if it falls inside a data range, and deduplicates exact tuples.

3.3 Messagebox worked example

The linear sweep only reaches `0x116` in this environment because Capstone 5.0.1's `skipdata` stops at the first undecodable byte. The three getpc sites are still visible:

Call site	Bytes	Callee	Pushed value	Data
`0xda`	`e8 0b 00 00 00`	`0xea: pop rcx`	`0xdf`	`"user32.dll"`
`0xfa`	`e8 11 00 00 00`	`0x110: pop rdx`	`0xff`	`"Hello, from MSF!"`
`0x111`	`e8 0b 00 00 00`	`0x121: pop r8`	`0x116`	`"MessageBox"`

The static scan produces:

```
(0xda, 0xdf, 11)
(0xfa, 0xff, 17)
(0x111, 0x116, 11)
```

No RIP-relative or immediate references exist in this payload, so these are the only three.

3.4 False positives and false negatives

Known false-positive classes:

- **Data decoded as code**: eliminated by the `overlaps_data` filter.
- **Branch/loop targets**: eliminated by the control-flow filter.
- **Constants that happen to fall inside a data range**: still possible for `mov`/`push` immediates. Example from another payload: `cmp word ptr [rax+0x18], 0x20b` targets the PE32+ magic `0x20b`, which can coincide with an address inside a large data string. The xref scan reports it; `extract_data()` classifies it as `imm`, but only `mov`/`movabs`/`push` instructions are treated as address materialization, so a `cmp` reference makes the section "unsupported" and it is left in place.
- **Absolute memory operands**: only valid for non-PIE code; reported if they land in a data range.

Known false negatives:

- Register-indirect pointers (`mov rax, rbp`, `lea rax, [rbx+0x10]`) require data flow, which the scan does not perform.
- References in code the linear sweep did not reach (Capstone stop, jump tables, code after invalid bytes).
- References from data (jump tables, pointer arrays) are not followed.

Every xref is therefore a candidate, not a proof of a semantic data access.

4. Data movement and patching (`--extract-data`)

4.1 Candidate selection

`extract_data()` (`pobf.py:1297`) receives `sections` and `xrefs`. It:

1. drops zero-length and out-of-range sections;
2. skips `outside_cfg` unless `include_unknown` (`--extract-unknown`);
3. sorts by offset and rejects overlapping candidates;
4. maps every xref to the candidate containing its target;
5. classifies each reference with `_classify_data_ref()`.

`_classify_data_ref()` accepts:

Kind	Condition
`getpc`	`call`; target == `call.end`; target == section start; callee is `pop r64`; `pop.address == section.start + section.length`; register in `reg_list64`
`rip`	`MEM` with `base == RIP`; `insn.address + insn.size + disp == target`
`abs`	`MEM` with no base/index; `disp & mask == target`
`imm`	`IMM` and mnemonic is `mov`/`movabs`/`push`; `imm == target`
`unsupported`	anything else

Sections with an unsupported reference, multiple getpc sites, a getpc region smaller than 7 bytes, overlapping regions, or an instruction crossing their boundary are **kept in place** and reported. This is the main safety valve: the tool prefers leaving data where it is over emitting a broken payload.

Candidate kinds and default treatment:

`kind`	Default	Rationale
`ascii`	move	printable run with NUL; call/pop gap corroboration when it overlaps CFG
`utf16le`	move	same, 16-bit encoding
`cfg_data`	move	CFG-inferred data object (`cfg.model.memory_data`)
`outside_cfg`	skip	unexecuted **and** not recovered by CFGFast; can be unreachable code
no xref	move	NOP the original bytes and append; nothing to patch

4.2 The getpc rewrite

For each valid getpc site, the whole `[call, data, pop]` region is replaced by a same-size `lea` plus NOPs:

```
before (17 bytes):
  0xda: e8 0b 00 00 00        call 0xea
  0xdf: "user32.dll\0"        11 bytes of data
```

```
  0xea: 59                    pop rcx

after (17 bytes):
  0xda: 48 8d 0d 5f 00 00 00  lea rcx, [rip + 0x5f]
  0xe1: 90 ... (10 NOPs)
```

Semantics:

- `call` pushes `0xdf`; `pop rcx` loads it into `rcx`.
- `lea rcx, [rip+disp]` loads the relocated data address into `rcx`.
- Neither `lea` nor `pop` changes flags.
- The call/pop pair has net-zero stack effect; the lea does not touch the stack. The visible register state and stack pointer at the next instruction are the same. (The transient 8 bytes below `rsp` are no longer written; code that reads that dead slot would observe a difference. This is documented as a caveat.)
- Control flow falls through the NOP padding to the instruction after the original `pop`, exactly as the original call/pop pair ended up there.

The region length must be at least 7 bytes. The placeholder is assembled with displacement 0; the final displacement is filled in later.

For a section with no getpc, the bytes simply become NOPs. This preserves the payload length and all instruction addresses.

4.3 Why size-preserving?

Compacting the code would shift every instruction after the data. The label table is built against the original address space and uses `instr_dict[target]`; a full renumbering would require rebuilding that mapping for every jump and call. The size-preserving rewrite avoids the problem:

- every original instruction address remains valid;
- `create_label_table()` / `update_label_table()` continue to work;
- only the data displacements need a second pass after morphing.

The cost is NOP padding in the code area. The data bytes themselves are gone from the code; they live in the appended blob.

4.4 Rebuild

After writing the lea/NOP bytes, `rebuild_instructions_from_bytes()` re-runs the linear sweep. This is necessary because the old sweep had decoded the strings as instructions; the boundaries around the getpc regions are now different.

The rebuild also recomputes `sweep_coverage` and `sweep_tail`. If the new disassembly does not cover the entire payload, extraction aborts:

```
if self.sweep_coverage != len(self.raw_code):
    raise RuntimeError("... refusing to extract data from a partially "
                       "disassembled payload")
```

The full-coverage requirement matters in this environment because Capstone 5.0.1's `skipdata` is broken; the original sweep stops at `0x116` (282 of 313 bytes). Removing the inline data makes the tail decodable, so the messagebox rebuild covers 313/313. Other payloads with remaining invalid bytes will abort instead of silently losing the tail.

4.5 Fixup registration

After the rebuild, the fixups are attached to the *new* instruction objects:

- getpc: the instruction at `region_start` must be a 7-byte `lea`;
- rip/abs/imm: the instruction at `source` must still encode the old target (`_check_rebuilt_reference`).

Each fixup records:

```
kind, instr, section, delta, source,
reg                         # getpc
op_index, old_disp          # rip/abs
op_index, old_imm           # imm
```

The wrapper object is tagged with `instr.data_ref_fixup = fixup`, so it can be found even if a morph replaces the Capstone object.

4.6 Finalization after morphing

`finalize_data_refs()` (`pobf.py:1609`) is called after every morphing step and before writing.

Step 1 — authoritative final addresses. It walks the current instruction list and accumulates byte lengths:

```
address = 0
for instr in self.instructions:
    addresses[id(instr)] = address
    address += len(instr.new_instruction.bytes)
code_length = address
```

This deliberately does not trust `instr.new_instruction.address`, because morphing may have changed sizes and not every path calls `update_instr()`. The cumulative walk is exactly the byte layout that `write_file()` will emit.

Step 2 — data layout. The blob starts at the next 16-byte boundary:

```
pad = (-code_length) % 16
cursor = code_length + pad
for section in self.extracted_data:
    section["new_address"] = cursor
    cursor += section["length"]
```

Sections keep their original order. Each reference target is `section["new_address"] + fixup["delta"]`.

Step 3 — patch getpc. The final displacement is

```
disp = target - (lea_address + 7)
```

Keystone assembles `lea reg, [rip ± disp]`; the instruction is re-decoded with Capstone so its operands are consistent.

Step 4 — patch RIP-relative / absolute memory. The old displacement hex is replaced in `insn.op_str` with the new one; the sign token (`rip +` / `rip -`) is normalized, the instruction is re-assembled at its final address, and the new length must equal the old length. For absolute memory the new displacement is the target offset directly.

Step 5 — patch immediates. The first hex literal in `op_str` is replaced with the target, the instruction is re-assembled, and the size must be unchanged. Only `mov`/`movabs`/`push` were classified as immediates, so this is limited to instructions that can materialize an address.

Step 6 — verification. Every patched reference is decoded and checked:

- getpc: mnemonic is `lea`, operand is `[rip + disp]`, register matches the original `pop` register, and `address + size + disp == target`;
- rip: `address + size + disp == target`;
- abs: `disp & mask == target`;
- imm: `op.imm == target`.

The data blob is also checked byte-for-byte against each section, and the engine warns if a moved section's bytes still appear in the code area.

4.7 Writing

`write_file()` (`pobf.py:2588`) emits:

```
[ code bytes ][ NOP pad ][ data blob ]
```

The data blob is not part of the instruction list. The output is longer than the input by `pad + len(blob)`.

For the non-extraction path, `write_file()` preserves an un-disassembled tail when no morphing was applied (`sweep_tail`), so a plain run reproduces the input exactly even when Capstone stops early. After morphing without extraction, the tail cannot be placed correctly; the engine prints a warning and the tail is omitted.

4.8 Messagebox results

Unmorphed:

```
code 313 B + pad 7 B + data 39 B = 359 B
data 0xdf -> 0x140 (11 B)   "user32.dll"
```

```
data 0xff -> 0x14b (17 B)   "Hello, from MSF!"
data 0x116 -> 0x15c (11 B)  "MessageBox"
ref 0xda  lea rcx, [rip + 0x5f] -> 0x140
ref 0xfa  lea rdx, [rip + 0x4a] -> 0x14b
ref 0x111 lea r8,  [rip + 0x44] -> 0x15c
```

After `-eq`:

```
code 330 B + pad 6 B + data 39 B = 375 B
data 0x150, 0x15b, 0x16c
lea sites 0xe7, 0x107, 0x11e
```

After `-nop`:

```
code 451 B + pad 13 B + data 39 B = 503 B
data 0x1d0, 0x1db, 0x1ec
lea sites 0x13b, 0x163, 0x18b
```

In all three cases:

- the three lea targets land exactly on the three strings;
- the strings no longer occur anywhere in the code region;
- `loop`/`jrcxz` displacements remain within the signed 8-bit range;
- every call/jmp/jcc target lands on a valid instruction start.

These checks are static (Capstone decoding, target arithmetic, byte comparison). The shellcode was not executed; a loader/runtime test is still the only way to prove end-to-end behavior on Windows.

4.9 Interaction with morphing

`equal_instructions` (`-eq`) changes instruction forms (xor → mov, add ↔ sub, etc.) and therefore sizes. It does not touch the `lea` placeholders, because none of its substitution patterns match `lea`. The finalization walk absorbs the new addresses.

`insert_random_nop` (`-nop`) inserts `pushf; <no-op>; popf` sequences after random instructions, including after a `lea`. The inserted templates preserve the target register and flags, so the pointer remains valid. Addresses shift, and finalization recomputes every displacement.

A new protection was added in `insert_random_nop()` (`pobf.py:702`): the `loop`/`jrcxz` span is computed before insertion and no NOP is inserted at a point that would change the branch's signed 8-bit displacement. The messagebox `loop 0x30` and `jrcxz 0xcd` therefore keep their distances (`-17` and `88`) instead of hitting the old `DIFF > 120` restart path.

`update_instr` / `update_label_table` handle control-flow references. They know nothing about data references; the `lea` placeholder has `disp=0` and is not in the label table. `finalize_data_refs()` is the only place that writes data displacements, and it runs after the last `update_label_table()`.

`brain_obfuscator` **(`-pii`)** is a different layout model (stub, instruction mapping, shuffle). RIP-relative data references would need the stub's mapping, so `--extract-data` refuses to combine with `-pii`.

Other techniques (`-bcf`, `-ibp`) insert or move code in ways that are not covered by the current verification. They are not claimed to work with extraction; the request explicitly prioritized the basic techniques.

4.10 CLI integration

`start()` (`pobf.py:2653`) maps the options to three modes:

- **`--find-data` / `-fd`**: `analyze_data()` (symbolic sections + xrefs), print, exit; no payload is written.
- **`--extract-data` / `-xd`**: `analyze_data()`, `extract_data()`, optional `create_label_table()` + morph + `update_label_table()`, then `finalize_data_refs()` and `write_file()`.
- **no data flag**: the normal pipeline runs; there is no symbolic analysis and no data movement. A plain write reproduces the input exactly (the `sweep_tail` path preserves any tail the linear sweep did not reach).

Additional switches:

- `--extract-unknown` / `-xu`: also consider `outside_cfg` ranges.
- `--data-timeout <seconds>`: passed to `find_data_sections()`.
- `--extract-data` + `--position-independent-instr`: rejected before analysis.

---

5. Failure modes and safeguards

Condition	Behavior
getpc does not exactly bracket the section	section kept in place, reported
pop register is `rsp`/`rbp` or 32-bit	not a supported getpc; kept in place
getpc region smaller than 7 bytes	kept in place
two getpc regions overlap	extraction aborts for that run
a section overlaps another candidate	owner kept in place, reported
an instruction crosses a non-getpc section boundary	section kept in place
unsupported reference (indexed memory, `cmp` immediate, etc.)	section kept in place
post-rewrite disassembly does not cover the payload	extraction aborts
a fixup instruction disappeared during morphing	finalization raises
patched reference fails verification	finalization raises before writing
data blob byte mismatch	finalization raises
`--extract-data` + `-pii`	rejected
`--extract-data` on a non-zero-based instruction space	rejected

The common theme: every uncertain case degrades to "leave the data where it is", not "write something that might be wrong".

---

6. Limitations and future work

1. **Symbolic coverage is not proof.** The environment has no PEB/TEB and no APIs. Observed instruction addresses are evidence, not reachability proof. The report exposes the stop reason and pending states for this reason.
2. **No data-flow xrefs.** A `mem_read` breakpoint or a register-value trace at `call rbp` would let the symbolic run corroborate data pointers. The current xref scan is static.
3. **`outside_cfg` is ambiguous.** The extractor skips it by default; moving it requires `--extract-unknown` and accepts the risk.
4. **Size-preserving rewrite leaves NOP holes.** Compaction requires a full relocation pass over every instruction and branch; the current design deliberately avoids it.
5. **The data blob is appended and must be loaded in full.** The output is larger than the input. Code that falls through the end will execute the NOP pad and then the blob; the original payload already had no terminator, so this does not make a well-formed return path worse, but it should be considered.
6. **The call/pop rewrite removes the transient stack write.** The net stack and register effects are identical, but code that reads the dead 8 bytes below `rsp` would observe a difference. This is very unusual, but it is a semantic difference.
7. **Immediate references are best-effort.** Only `mov`/`movabs`/`push` are treated as address materialization; a computed address (`add reg, imm`) is not rewritten.
8. **Capstone version sensitivity.** The code expects 5.0.0.post1. In this environment (5.0.1) `skipdata` stops the linear sweep at the first invalid byte. The extractor rebuilds after removing data and aborts if coverage is still incomplete; the plain path preserves the unscanned tail only when no morphing is applied.
9. **Only x64 raw payloads.** `find_data_and_refs()` is hardcoded to `x64` and payload-relative addresses. `pobf86.py` has no equivalent data extractor.

---

7. Appendix: key code locations

Function	File	Line
`ranges_not_in_cfg`	`symbolic_payload_coverage.py`	63
`static_candidates`	`symbolic_payload_coverage.py`	87
`identify_data`	`symbolic_payload_coverage.py`	119
`find_data_sections`	`symbolic_payload_coverage.py`	254
`analyze` (base rebasing)	`symbolic_payload_coverage.py`	319
`explore`	`symbolic_payload_coverage.py`	388
`Instruction`	`pobf.py`	22
`Zone.__init__`	`pobf.py`	38
`update_instr`	`pobf.py`	239

Function	File	Line
`update_label_table`	`pobf.py`	290
`insert_random_nop` (short-branch guard)	`pobf.py`	702
`find_data_and_refs`	`pobf.py`	985
`rebuild_instructions_from_bytes`	`pobf.py`	1160
`assemble_lea_rip`	`pobf.py`	1189
`analyze_data`	`pobf.py`	1203
`_classify_data_ref`	`pobf.py`	1235
`extract_data`	`pobf.py`	1297
`_patch_memory_reference`	`pobf.py`	1530
`_patch_immediate_reference`	`pobf.py`	1558
`_verify_data_ref`	`pobf.py`	1577
`finalize_data_refs`	`pobf.py`	1609
`write_file` (code + pad + data)	`pobf.py`	2588
`start` (CLI)	`pobf.py`	2653